ITI1121 – Introduction To Computing

Variables

Definition 1 (Variables)

A variable is a place in memory to hold a value, which we refer to using a label.

```
1    byte i = 33;
```

Data Types in Java

- Java is a strongly typed programming language. Each variable has a size and a known type at compile time.
- The type of each variable must be declared.

```
1 int i;
2 i = 33;
```

Big Question 1 (Data Types)

Q: What are data types for?

A:

- Data types tell the compiler how much memory to allocate.

  Example: Instantiation of a double data type tells the compiler to allocate 8 bytes.

- Data types also give information about which operations are allowed.

  Example: When a char is declared, the compiler knows to treat it as a char. So it cannot be multiplied by a number, etc. (operations).

- Java has both primitive and reference data types.

  Definition 2 (Primitive Data Types)

  A primitive data type specifies the type of a variable and the kind of values (stored at memory location) it can hold.

  Definition 3 (Reference Data Types)

  A reference data type references/points to the location of an object.

  - User-defined, and refers to an instance of a class.

- The value of a reference variable is a memory location which points to the location of an object.

- The declaration of a reference variable only allocates memory to store the address/location of an object.

  - These objects contain functions which return values.

- Declaring a reference variable allocates memory for the location; `null` is when there is no object being referred to.

Memory Diagrams

Below shows code and its memory diagrams.

```
1 public class Constant {
2     private String name;
3     private double value;
4     public Constant( String name, double value) {
5         this.name = name;
6         this.value = value;
7     }
8 }
```

```
1 Constant c;
2 c = new Constant("Golden ratio", 1.61803399)
```

Reference variable of type Constant

Address	Value
c	

Constant Class Instance

Address	Value
value	1.61803399
name	

String Class Instance

Address	Value
	"Golden ratio"

Reference variable, c, is pointing to the Constant Class, which has an int variable and a String variable. But since String is also a reference type it points to the String class.

Classes

Definition 4 (Auto-Boxing)

Auto boxing is when an object is declared, but is made equal to its primitive version. Java 5 and up will auto-box it and compile, Java 1.4 and under will not.

```
Integer i = 1;
```

Java 5 treats it as:

```
Integer i = Integer.valueOf(1);
```

Java 5's implementation was deprecated since Java 9 and is now just:

```
Integer i = new Integer(1);
```

Important: Auto-Boxing will seriously affect runtime

Comparison Operators

Primitive variables can be compared using basic operators such as $<$, $>$, $=$ etc. Reference variables require dedicated functions to compare the values of objects.
In cases where technically, a basic operator compares a reference variable to reference variable, it will treat them as different objects.

```
MyInteger a = new MyInteger(5);
MyInteger b = new MyInteger(5);

if (a == b) {
    System.out.println("a equals b");
} else {
    System.out.println("a does not equal b"); \\fuckin latex format
}
```

This code will print as "a does not equal b" as Java is comparing two refrences which's value is different at memory location.
This implies that if two objects point to the same object (same memory location) the latter wouldnt have printed, but rather "a equals b".

Call-by-Value

Definition 5 (Formal Parameter)

A formal parameter is a variable which is part of the definiition of the method

Definition 6 (Actual Parameter)

The actual parameter is the real input values of the method when the method is called

```
int sum(int a, int b) { //formal parameters a, b
    return a + b;
}

int x,y,z;
total = sum(x,y); //actual parameters x,y
```

In Java, when a method is called the values of the actual parameters are copied to the location of the formal parameters. Below is what happens when a function is called.

1. the execution of the calling method is stopped
2. an activation frame (activation block or record) is created (it contains the formal paramerters as well as the local variables)
3. the value of the actual parameters are copied to the locaiton of the formal parameters
4. the body of the method is executed
5. a return value or void is saved
6. activation frame is destroyed
7. the execution of the claling method restarts with the next instruction

Example:

```
    public static void increment(int a) {
        a = a + 1;
    }
    public static void main( String[] args) {
        int a = 5;
        increment(a);
    }
```

the increment function doesnt actually return any real values so step 5 is skipped and the activation frame for increment(a) is destroyed with no real outcome on the main function.